

Converted from: AI (III CSE) - Unit 3 Notes.docx

Artificial Intelligence

Unit 3 – Representation of Knowledge

Representation of Knowledge: Knowledge representation issues, predicate logic- logic programming, semantic nets- frames and inheritance, constraint propagation, representing knowledge using rules, rules-based deduction systems. Reasoning under uncertainty, review of probability, Bayes' probabilistic interferences and Dempster-Shafer Theory theory.

Major issues in knowledge representation in AI

Why Knowledge Representation is Important for Problem Solving:

What is Knowledge Representation? A way to store facts and rules so a computer can think and act.

Turn problems into data: Computers can search that data to find answers. Example: Cities = dots, roads = lines !' find the shortest path.

Make new facts from old ones (inference): If we know "Tweety is a bird" and "birds can fly," we get "Tweety can fly."

Handle limits and choices (constraints): Stops bad options. Example: Class size must be "d room capacity when making a timetable.

Explain answers: Shows why a result was chosen. Example: "Diagnosis because symptoms A, B, C matched Rule-3."

Share common meanings: Different systems agree on terms. Example: A medical ontology makes "BP" mean the same thing everywhere.

Major Issues in Knowledge Representation in AI

Incomplete Knowledge

Real-world knowledge is often partial or missing.

Example: A medical expert system may not have all the symptoms for a rare disease.

Explanation: The system doesn't know everything, so it can miss the correct diagnosis.

Uncertainty

Information may be imprecise, vague, or probabilistic.

Example: A patient may have fever, but the degree and cause are uncertain.

Explanation: We aren't fully sure what's true, so conclusions can only be "likely," not certain.

Scalability / Complexity

Large knowledge bases are difficult to maintain, update, and process efficiently.

Example: Representing all facts about natural language semantics is extremely complex.

Explanation: There's too much to store and manage, so the system becomes heavy and slow.

Consistency

Contradictory knowledge may exist.

Example: "All birds fly" vs. "Penguins are birds that don't fly."

Explanation: Rules can clash; one rule says "yes," another says "no."

Reasoning Efficiency

Even if knowledge is represented well, reasoning over it must be efficient.

Example: Logic-based systems may take too long to infer answers in real time.

Explanation: The system might be correct but too slow to be useful.

Context Dependence

Meaning of knowledge may change depending on context.

Example: The word “bank” could mean a financial institution or riverbank.

Explanation: The same word can mean different things; the system must know the situation.

Knowledge Acquisition Bottleneck

Extracting, formalizing, and updating knowledge is slow and resource-intensive.

Example: Encoding expert medical rules into an AI system requires domain experts.

Explanation: Getting expert knowledge into the system takes a lot of time and people.

Expressiveness vs. Computability Trade-off

More expressive representations (like natural language) are harder to process, while more structured ones (like logic) may be less flexible.

Explanation: If it's easy for humans, it's hard for machines; if it's easy for machines, it's less natural for humans.

Note: Knowledge representation is at the core of AI problem solving. Without it, AI cannot store, interpret, or reason about facts to solve real-world problems. But designing a representation that is complete, consistent, efficient, and scalable remains a major challenge.

Case Study: How Knowledge Representation helps solve a problem

Task: Simple illness guess (cold Vs. flu)

Representation (IF-THEN rules):

R1: Fever " BodyAche !" Flu

R2: RunnyNose " Sneezing !" Cold

R3 (default): Fever !" LikelyFlu (unless evidence against)

Facts (patient P): Fever, SneezingReasoning:

From R3: LikelyFlu(P) (default).

From R2 needs RunnyNose AND Sneezing. We only know Sneezing. !' Can't conclude Cold.

Incomplete knowledge issue: Not knowing RunnyNose blocks rule R2.

If new fact arrives: BodyAche !' Use R1 to confirm Flu.

If we later learn P is a penguin (even at age 2), we take back the default "P can fly."
That's non-monotonic reasoning.

Predicate logic in AI and apply Logic Programming

What is Predicate Logic (First-Order Logic, FOL)?

A formal way to write facts and rules about the world so a machine can reason.

Building blocks

Constants: specific objects — alice, nepal, cs101

Variables: placeholders — X, Y

Predicates: properties/relations — Parent(alice,bob), Enrolled(bob,cs101)

Functions (optional): build new objects — FatherOf(bob)

Connectives: ""(and), ""((or), !'(implies), ¬(not)

Quantifiers: " X (for all X), " Y (there exists Y)

Example (English !' FOL)

"All humans are mortal." !' " x (Human(x) !' Mortal(x))

"Socrates is a human." !' Human(socrates)

From these, conclude: Mortal(socrates).

Why use it? Clear meaning, supports inference, and scales better than plain propositional logic.

From FOL to Logic Programming (Prolog-style):

Logic programming uses a restricted form of logic called Horn clauses:

Fact:parent(alice, bob).

Rule:grandparent(X, Z) :- parent(X, Y), parent(Y, Z).("X is Z's grandparent if X is parent of Y and Y is parent of Z.")

Query/Goal: ?- grandparent(alice, Z).

How it works

Unification: match patterns by finding variable bindings (X=alice, Y=bob, ...).

SLD-resolution & backtracking: try rules left to right; if a path fails, backtrack to try alternatives.

Closed-world assumption & negation-as-failure: if Prolog can't prove it, it assumes false (for that query).

Example – 1: Step-by-Step (Family Relations):

Knowledge Base (facts + rules)

% Facts

parent(alice, bob).

parent(bob, cara).

parent(alice, dan).

parent(dan, emma).

female(alice).

male(bob).

male(dan).

% Rules

mother(M, C) :- parent(M, C), female(M).

father(F, C) :- parent(F, C), male(F).

grandparent(G, K):-parent(G, P), parent(P, K).

ancestor(A, D) :- parent(A, D).

ancestor(A, D) :- parent(A, X), ancestor(X, D).

Query 1: Who are Alice's grandchildren?

?- grandparent(alice, K).

Derivation (informal trace)

Goal: grandparent(alice, K)

Use rule: grandparent(G,K) :- parent(G,P), parent(P,K)

Unify G = alice !' subgoals: parent(alice,P) and parent(P,K)

From facts, parent(alice,bob) and parent(alice,dan):

Case A: P=bob !' need parent(bob,K) !' from facts parent(bob,cara) !Ò K=cara

Case B: P=dan !' need parent(dan,K) !' from facts parent(dan,emma) !Ò K=emma

Answers: K = cara ; K = emma. (with backtracking)

Query 2: Who is Cara's father?

?- father(F, cara).

Derivation

father(F,C) :- parent(F,C), male(F)

From facts: parent(bob,cara) and male(bob) ! $\Rightarrow$ F = bob.

Example – 2: A Tiny Rule-Based Eligibility (to show inference chain)

Knowledge Base

% Facts

student(rahul).

gpa(rahul, 8.3).

income_family(rahul, low).

attendance(rahul, 0.92). % 92%

% Helper predicates

good_gpa(S) :- gpa(S,G), G >= 8.0.

high_attendance(S) :- attendance(S,A), A >= 0.85.

% Scholarship rule

eligible_scholarship(S) :-

student(S),

good_gpa(S),

high_attendance(S),

income_family(S, low).

Query

```
?-eligible_scholarship(rahul).
```

Reasoning

```
student(rahul) '
```

```
gpa(rahul,8.3) !Ògood_gpa(rahul) '
```

```
attendance(rahul,0.92) !Òhigh_attendance(rahul) '
```

```
income_family(rahul,low) '
```

All subgoals satisfied !ÒRahul is eligible.

Semantic Networks

A semantic network is a graph of concepts (nodes) connected by labeled links (edges) that describe how those concepts are related.

It's used in AI for storing meaning, inheritance (passing properties down a hierarchy), and explainable reasoning.

A semantic network represents knowledge as a graph:

Nodes = objects/concepts (e.g., Bird, Penguin, Wing).

Labeled edges = relationships (e.g., is-a, has-part, instance-of, can).

It's great for:

Modeling hierarchies (is-a links for inheritance)

Showing part-whole (has-part)

Capturing abilities/properties (can, lives-in)

Doing simple reasoning by following links.

A semantic network is a diagram that uses circles for concepts and arrows for relationships (e.g., is-a, instance-of, has-part), enabling property inheritance and exception handling, as shown in the figure below.

Figure: Semantic Network

The diagram is a semantic network where circles are concepts or specific examples and arrows show relationships.

Bird and Fish are both “is-a” kinds of Animal, and Penguin “is-a” Bird, so a penguin is also an animal.

Tweety is an “instance-of” Penguin and Nemo is an “instance-of” Fish, meaning they inherit their class properties.

Class properties are attached by labeled links: Bird “has-part” Wing and, by inheritance, penguins also have wings; Penguin additionally “has-part” Feather.

Birds generally “can” Fly, but the dashed “cannot” link from Penguin to canFly marks an exception, so Tweety (a penguin) does not fly. Fish “lives-in” Water, so Nemo lives in water.

How relationships are represented

is-a (subclass)

Bird !' Animal (is-a), Penguin !' Bird (is-a)

Meaning: every Penguin is a Bird, and every Bird is an Animal.

instance-of (object)

Tweety !' Penguin (instance-of)

Tweety is a particular penguin (an object, not a class).

has-part (part-whole)

Bird !' Wing (has-part), Penguin !' Feather (has-part)

Birds have wings; penguins have feathers.

can / cannot (ability/constraint)

Bird !' CanFly (can) encodes the typical ability.

Penguin !' CanFly (cannot) (dashed) encodes the exception: penguins don't fly.

lives-in (relation to environment)

Fish !' Water (lives-in)

Class-level ecological fact.

Step-by-step reasoning example (inheritance + exception)

Query: "Can Tweety fly?"

Step 1: Follow Tweety !' Penguin (instance-of) !' check Penguin.

Step 2: Penguin has a cannot link to CanFly !' conclude Tweety cannot fly.

(If the exception were absent, we'd inherit the typical ability via Tweety !' Penguin !' Bird !'

CanFly (can) and answer yes.)

Query: "Is Tweety an animal?"

Tweety (instance-of) Penguin !' (is-a) Bird !' (is-a) Animal

By transitivity of is-a, yes.

Query: "What parts does a Penguin have?"

Direct: Penguin !' Feather (has-part)

Inherited from Bird: Bird !' Wing (has-part) !Ò Penguins have wings (even if they don't fly).

Semantic Network View:

Bird --has-part--> Wing

Bird --has-part--> Feather

Fish --lives-in--> Water

Bird --can--> CanFly

Penguin --cannot--> CanFly

A semantic net is a labeled graph. Inference follows the links (with inheritance and exceptions) to reason about objects and their relationships.

Frame-based Knowledge Representation

Frame-Based Knowledge Representation:

Frames are like templates for things (objects, classes, situations).

Each frame has slots (properties) with values.

Frames can also have defaults, rules/limits, and small procedures that fill a value when needed.

A frame can point to a parent frame and inherit its properties.

Why use frames?

Easy to model real-world hierarchies (like Animal !' Mammal !' Dog).

Keep related facts together in one place.

Support defaults and exceptions (typical case + special cases).

Combine facts with small procedures (how to compute a value).

What's inside a frame?

Name: what the frame is about (e.g., Dog, sheru).

Slots: properties (e.g., has_legs, diet, sound).

Slot details (optional): default value, allowed types, constraints, small procedure.

Links:

is-a: instance of a class (sheru is-a Dog)

ako ("a kind of"): subclass link (Dog ako Mammal)

part-of: component link (e.g., Wheel part-of Car)

How inheritance works (the basic idea)

When you need a slot's value:

Check the instance frame first.

If missing, check its class frame.

If still missing, go up the parents (via ako/is-a) until you find a value.

Closer values override higher-level defaults.

If there are multiple parents, use a clear rule to break ties (order or an explicit policy).

Worked example: Animals !' Mammals !' Dogs (with an exception)

1) Class hierarchy

Thing

 % % % Animal

 % % % Bird

 % % % Mammal

 % % % Dog

 % % % Bat (a Mammal that can fly)

2) Frames (compact notation)

Frame: Animal

slots:

has_cells := true (default)

blood_type := warm_blooded (default)

movement := {walk, run, swim} (typical)

can_fly := false (default)

diet := varies (default)

Frame: Bird aka !' Animal

slots:

has_feathers := true

lays_eggs := true

can_fly := true (default, with exceptions like Penguin)

Frame: Mammal aka !' Animal

slots:

has_hair := true

gives_milk := true

lays_eggs := false

can_fly := false (reinforces Animal default)

Frame: Dog ako !' Mammal

slots:

sound := "bark"

diet := carnivore (overrides Animal's vague default)

Frame: Bat ako !' Mammal

slots:

can_fly := true (exception overriding Mammal default)

diet := insectivore (typical)

Frame: sheru is-a !' Dog (an instance)

slots:

color := brown

(no explicit can_fly, has_hair, etc.)

3) Inheritance in action (query !' answer)

Q1. Does sheru have hair? Lookup order:

sheru.has_hair? (missing) !' look at class Dog (missing) !' parent Mammal.has_hair = true. A: true (inherited from Mammal).

Q2. Can sheru fly?

sheru.can_fly? (missing) != Dog.can_fly? (missing) != Mammal.can_fly = false.A: false (inherits Mammal default).

Q3. What's sheru's diet?

sheru.diet? (missing) != Dog.diet = carnivore (class override over Animal's vague default).A: carnivore.

Q4. Can a Bat fly? Why is this an exception?

Bat.can_fly = true overrides Mammal.can_fly = false.A: true due to a specific subclass override (exception handling).

Multiple inheritance:

Suppose RobotDog inherits from both RobotPet and Dog:

Frame: RobotPet

power_source := battery

sound := "beep"

Frame: RobotDog aka != {Dog, RobotPet}

conflict on sound: Dog.sound = "bark" vs RobotPet.sound = "beep"Resolution options:

Precedence order in parents (e.g., prefer Dog over RobotPet) != sound = "bark".

Explicit override in RobotDog (e.g., set sound = "bark-beep").

Policy: "Prefer most specific, else first-listed parent."

Constraint Propagation to a suitable Constraint Satisfaction Problem (CSP)

It cuts down choices early, so CSPs solve faster with less backtracking.

It's used in real tasks like timetables, Sudoku, routing, and scheduling.

Constraint Satisfaction Problem (CSP): set of variables, each with a domain of values, and constraints that restrict which combinations of values are allowed.

Constraint propagation: automatic pruning of domain values that cannot participate in any solution, thus reducing search.

CSP Setup: India Map-Coloring

Variables (states/UT): GJ (Gujarat), RJ (Rajasthan), MP (Madhya Pradesh), MH (Maharashtra), UP (Uttar Pradesh), HR (Haryana), DL (Delhi)

Domains (initial): each {R, G, B} (Red, Green, Blue)

Constraints (adjacent regions must be different): GJ–RJ, GJ–MP, GJ–MH, RJ–MP, RJ–HR, RJ–UP, HR–DL, HR–UP, UP–DL, MP–UP, MP–MH (DL borders HR & UP; no direct RJ–DL or DL–MP)

Step 0 — Initial Domains

GJ:{R,G,B} RJ:{R,G,B} MP:{R,G,B}

MH:{R,G,B} UP:{R,G,B} HR:{R,G,B} DL:{R,G,B}

Step 1 — Assign GJ = R, then Forward Check

Decision: GJ = R

Neighbors of GJ: RJ, MP, MH must be $\neq$ R

RJ:{G,B} MP:{G,B} MH:{G,B}

UP:{R,G,B} HR:{R,G,B} DL:{R,G,B}

Step 2 — Assign RJ = G, then Forward Check

Decision: RJ = G (from {G,B})

Neighbors of RJ: GJ, MP, HR, UP

GJ = R (ok)

MP " G !" MP:{B} (forced)

HR " G !" HR:{R,B}

UP " G !" UP:{R,B}

State now:

GJ:R

RJ:G

MP:{B}

MH:{G,B}

UP:{R,B}

HR:{R,B}

DL:{R,G,B}

Step 3 — Propagate MP = B (forced), then Cascade

Assign: MP = B

Neighbors of MP: GJ, RJ, UP, MH

GJ = R (ok)

RJ = G (ok)

UP " B !' UP:{R} !ÒUP = R

MH " B !' MH:{G} !ÒMH = G

State:

GJ:R

RJ:G

MP:B

MH:G

UP:R

HR:{R,B}

DL:{R,G,B}

Step 4 — Propagate UP = R

Neighbors of UP: RJ, HR, DL, MP

RJ = G (ok)

MP = B (ok)

HR " R !' HR:{B} !ÒHR = B

DL " R !' DL:{G,B}

State:

GJ:R

RJ:G

MP:B

MH:G

UP:R

HR:B

DL:{G,B}

Step 5 — Propagate HR = B

Neighbors of HR: RJ, UP, DL

RJ = G (ok)

UP = R (ok)

DL " B and (already " R from UP) !' DL:{G} !DL = G

Final Coloring (all assigned after two choices!)

GJ = R

RJ = G

MP = B

MH = G

UP = R

HR = B

DL = G

Table — After Each Step

Step

GJ

RJ

MP

MH

UP

HR

DL

0 (initial)

{R,G,B}

{R,G,B}

{R,G,B}

{R,G,B}

{R,G,B}

{R,G,B}

{R,G,B}

1 (GJ=R + FC)

R

{G,B}

{G,B}

{G,B}

{R,G,B}

{R,G,B}

{R,G,B}

2 (RJ=G + FC)

R

G

{B}

{G,B}

{R,B}

{R,B}

{R,G,B}

3 (propagate MP=B)

R

G

B

{G}

{R}

{R,B}

{R,G,B}

4 (propagate UP=R)

R

G

B

G

R

{B}

{G,B}

5 (propagate HR=B)

R

G

B

G

R

B

{G}

Final

R

G

B

G

R

B

G

Conclusion:

Propagating constraints after each assignment removes impossible values and quickly fixes many variables.

In the India map example, just two choices forced all others—showing how simple and powerful this idea is.

How the Search Space is Reduced?

Start: 7 regions $\times$ 3 colors $\Rightarrow$ 2187 possibilities.

After GJ = R: neighbors lose R $\Rightarrow$ about 216 possibilities.

After RJ = G: more pruning (MP forced, others shrink) $\Rightarrow$ about 24.

Cascade (forced values): UP, MH, HR, DL get fixed ! → 1 final coloring.

Note:

Set one region's color !' its neighbors can't use that color.

A neighbor may have only one color left; we set it, which then limits its neighbors too — a domino effect.

Knowledge Representation using Rules

Knowledge Representation with Rules:

What is rule-based KR?

We store knowledge as IF–THEN rules plus facts.

Fact (working memory): a true statement now. Example: `student(rahul), gpa(rahul)=8.3`

Rule (knowledge base): pattern !' conclusion/action. Example: IF `student(X) AND gpa(X) >= 8.0` THEN `good_gpa(X)`

Inference engine applies rules to facts to infer new facts (deduction).

How inference works (two common styles)

Forward chaining (data-driven): Start from known facts, fire rules that match, keep adding conclusions until goal reached.

Backward chaining (goal-driven): Start from a query/goal, look for rules that could prove it, then try to prove their preconditions.

Example: Scholarship Eligibility (step-by-step)

Facts (given)

`student(rahul)`

`gpa(rahul) = 8.3`

attendance(rahul) = 0.92

family_income(rahul) = low

Rules (IF-THEN)

IF gpa(X) $\geq$ 8.0 THEN good_gpa(X)

IF attendance(X) $\geq$ 0.85 THEN high_attendance(X)

IF student(X) AND good_gpa(X) AND high_attendance(X) AND family_income(X)=low
THEN eligible_scholarship(X)

(optional disqualifier) IF attendance(X) < 0.75 THEN not_eligible(X)

Forward chaining trace:

Start WM (working memory):{ student(rahul), gpa(rahul)=8.3, attendance(rahul)=0.92,
family_income(rahul)=low }

Rule 1 matches gpa(rahul)=8.3 !' add good_gpa(rahul)WM !• ..., good_gpa(rahul)

Rule 2 matches attendance(rahul)=0.92 !' add high_attendance(rahul)WM !• ...,
high_attendance(rahul)

Rule 3 now fully matches (student, good_gpa, high_attendance, family_income=low)!'
add eligible_scholarship(rahul)

Rule 4 does not match (attendance is not < 0.75)

Conclusion:eligible_scholarship(rahul) is true (derived).

Backward chaining sketch (same goal)

Goal:eligible_scholarship(rahul)

Look at Rule 3 (it concludes the goal). Need to prove:

student(rahul) '

good_gpa(rahul) !' from Rule 1 needs gpa(rahul) $\geq$ 8.0 ' (8.3)

high_attendance(rahul) !' from Rule 2 needs attendance $\geq$ 0.85 ' (0.92)

family_income(rahul)=low ' All subgoals proved !' goal holds.

Reasoning under uncertainty in AI

What is “reasoning under uncertainty”?

It's how AI represents, updates, and uses incomplete or noisy information to make the best possible decision.

Instead of binary true/false facts, we carry degrees of belief (e.g., probabilities) and choose actions that maximize expected benefit (utility).

Why it matters?

Real data are messy: sensors fail, users omit details, tests have errors.

Avoids brittle yes/no logic: supports graded conclusions.

Enables rational action: pick the action with highest expected utility given current beliefs.

Quantifies value of information: decide when another test/measurement is worth it.

Common tools:

Probability & Bayes' rule (posterior = prior $\times$ likelihood / evidence)

Bayesian networks/HMMs, MDPs/POMDPs (decision making over time)

Also: fuzzy logic, Dempster–Shafer, credal sets (alternatives to classic probability)

Worked example: Medical test !' treat, don't treat, or test again?

Goal: Decide what to do after a positive screening test.

Given

Disease prevalence (prior): $P(D)=0.01$

Test-1: sensitivity 0.95, specificity 0.90 (false-positive 0.10)

Utilities (arbitrary units):

Treat & diseased: +100 (benefit)

Treat & healthy: -20 (side-effects/cost)

Don't treat & diseased: -100 (missed treatment)

Don't treat & healthy: 0

1) Update belief after the positive result (Bayes)

2) Decision threshold (treat Vs don't treat)

Our posterior is 8.76%, just below the threshold ! Ò don't treat yet.

3) Should we do a better second test?

Assume Test-2: sensitivity 0.99, specificity 0.99; test cost = -2 utility.

Current odds after Test-1: $0.0876 / 0.9124 = 0.096$.

If Test-2 positive: likelihood ratio $= 0.99 / 0.01 = 99$. New odds $= 0.096 \times 99 = 9.51$! Ò P " H 0.905. This is "k 9.09%, so we treat. EU " H 120(0.905) " 20 " H +88.6.

If Test-2 negative: LR $= 0.01 / 0.99$ " H 0.0101. New odds $= 0.096 \times 0.0101$ " H 0.0010 ! Ò P " H 0.001. Below threshold ! Ò don't treat. EU " H " 100(0.001) " H " 0.10.

Probability that Test-2 is positive given current belief:

$P(+2) = 0.0876 \times 0.99 + 0.9124 \times 0.01$ " H 0.0959.

Expected utility of "test again"

$EU(\text{test}) = -2 + 0.0959 \times 88.6 + 0.9041 \times (-0.10)$ " H +6.4.

Compare options now

Treat now: "H" 9.49

Don't treat now: " $100 \times 0.0876 = 8.76$

Do Test-2: "H+6.4 Ø>ÜjBest

Bayes' probabilistic inference

Bayes' inference updates what you believe before (prior) after seeing evidence (likelihood) to get a new belief (posterior).

Rule (Bayes' theorem):

$$P(H|E) = (P(E|H) P(H)) / (P(E|H) P(H) + P(E|\neg H) P(\neg H))$$

H: hypothesis, E: evidence

P(H): prior belief, P(E|H): likelihood, P(H|E): posterior belief

Example: If a disease is 1% ($P(H)=0.01$), test is 95% sensitive and 90% specific, then after a positive test the chance of disease is about 8.8%, not 95% (because positives can occur in healthy people).

Bayes' probabilistic inference — Example

Setup

Hypothesis H: person has the disease.

Prior (prevalence): $P(H)=0.01$ (1%).

Test sensitivity: $P(+|H)=0.95$.

Test specificity: $0.90 \Rightarrow P(+|\neg H)=0.10$ (false-positive rate).

Step 1 — Write Bayes' rule

$$P(H|+) = (P(+|H) P(H)) / (P(+|H)P(H) + P(+|\neg H) P(\neg H))$$

Step 2 — Plug in numbers

$$P(+|H)P(H) = 0.95 \times 0.01 = 0.0095.$$

$$P(+|\neg H)P(\neg H)=0.10\times 0.99=0.099$$

$$\text{Denominator} = 0.0095 + 0.099 = 0.1085.$$

Step 3 — Compute posterior

$$P(H|+) = 0.0095 / 0.1085 \approx 0.0876 \quad (\text{about } 8.8\%)$$

Meaning: Even with a good test, a positive result gives only ~8.8% chance of disease because the disease is rare (base-rate effect).

Frequency check

Out of 10,000 people: 100 sick !' 95 test positive; 9,900 healthy !' 990 false positives.

$$\text{Total positives} = 95 + 990 = 1,085.$$

$$\text{Posterior} = 95 / 1,085 \approx 8.8\% \text{ (same result).}$$

Odds form

$$\text{Prior odds} = 0.01 / 0.99 \approx 0.0101$$

$$\text{Bayes factor} = 0.95 / 0.10 = 9.5$$

$$\text{Posterior odds} = 0.0101 \times 9.5 \approx 0.0959$$

$$\text{Posterior prob} = 0.0959 / (1 + 0.0959) \approx 0.0876$$

Dempster–Shafer (D–S) — very simple contrast

Idea

Allows explicit ignorance. You can assign support (mass) to H , to $\neg H$, or to “unknown” (the whole set $\{H, \neg H\}$).

Output is an interval:

Belief $\text{Bel}(H)$: confirmed support.

Plausibility $\text{Pl}(H)$: how much H could still be true.

Always $\text{Bel}(H) \leq P(H) \leq \text{Pl}(H)$.

Example:

Sensor A: $m_1(H)=0.6$, $m_1(\theta, \text{“Ó ãB f C R –væ÷ ance})$.

Sensor B: $m_2(H)=0.3$, $m_2(\theta, \text{“Ó ãrà$

Combine (no conflict):

$$m(H)=0.6 \times 0.3 + 0.6 \times 0.7 + 0.4 \times 0.3 = 0.72$$

$$m(\theta, \text{“Ó ãB, S ãsÓ ã€$$

So $\text{Bel}(H)=0.72$, $\text{Pl}(H)=1.0$!' interval $[0.72, 1.00]$ shows remaining ignorance.

When to use which?

Bayes: you have priors + likelihoods and want a single posterior number.

D–S: evidence is incomplete/ambiguous and you want to keep ignorance explicit.

Bayes Vs. D–S — when to use what

Requires a prior?

Bayes: Yes (precise numeric prior).

D–S: No precise prior; can place mass on ignorance.

Output:

Bayes: Single posterior probability $P(H|E)$.

D–S: Interval $[\text{Bel}(H), \text{Pl}(H)]$ capturing certainty and ignorance.

Combining evidence:

Bayes: Multiply likelihoods (given modeling assumptions/independence).

D–S: Dempster's rule to fuse masses; handles partial/uncertain sources.

Use Bayes when you can state priors/likelihoods and want crisp posteriors.

Use D–S when evidence is incomplete/ambiguous and you want to represent uncertainty about the model itself (ignorance).